Practical 2

Spatial Filtering: Implementing 2D Convolution and Applying Image Kernels

---

Aim

To implement the 2D convolution (spatial filtering) operation from scratch using zero-padding, and to apply different filter kernels — Mean, Laplacian, Sharpening, Prewitt (gradient) and Gaussian — on a sample image matrix in order to observe their effects.

Conceptual / Theoretical Background

Spatial filtering (also called convolution or correlation in image processing) is a technique in which a small matrix called a kernel or mask (typically 3×3) is slid over every pixel of an image. At each position, the kernel values are multiplied element-wise with the underlying image pixels and summed to produce one output pixel. Repeating this across the whole image produces a new, filtered image.

Because the kernel needs neighbouring pixels on all sides, pixels at the image border do not have full neighbourhoods. Padding solves this by adding extra rows/columns (commonly filled with zeros) around the image so every original pixel, including the border ones, can be processed.

Different kernels produce different effects. A Mean (averaging) kernel, where every weight is equal, smooths/blurs the image by replacing each pixel with the average of its neighbourhood. A Laplacian kernel approximates the second derivative of intensity and highlights regions of rapid intensity change (edges). A Sharpening kernel is essentially the original pixel boosted and the Laplacian-like neighbourhood subtracted, which accentuates edges while keeping the rest of the image recognisable. A Prewitt-style kernel (weights arranged as -1/0/+1 rows) approximates the intensity gradient in one direction and highlights directional edges. A Gaussian kernel weights the centre pixel most heavily and neighbours progressively less, giving smoother, more natural blurring than a plain mean filter.

Because kernel weights can sum to a value other than 1 (e.g. the Mean and Gaussian kernels used here sum to 9 and 16 respectively before normalisation), the raw convolution output is divided by a scale factor so that the resulting pixel intensities stay in a comparable range to the input.

Real-World Applications

- Photo-editing and social-media apps use averaging/Gaussian kernels for blur effects and sharpening kernels for the 'sharpen' or 'clarity' tool.
- Medical image pre-processing uses smoothing filters to reduce scanner noise before diagnosis, and edge/Laplacian filters to highlight tissue or tumour boundaries.
- Satellite and aerial imagery uses sharpening filters to enhance fine details such as roads and buildings.
- Optical character recognition (OCR) pipelines apply smoothing to remove scan noise and sharpening to make character edges clearer before recognition.
- Feature extraction in classical computer vision (before deep learning) relies on gradient kernels such as Prewitt and Sobel to detect edges as a first processing stage.

Important Terminology

Term	Meaning
Kernel / Mask	A small matrix (e.g. 3×3) of weights that is convolved with an image to produce a filtering effect.
Convolution	The operation of sliding a kernel over an image and computing a weighted sum at each position.
Padding	Extra border pixels (here, zeros) added around an image so edge pixels can also be filtered.
Scale factor	A value the raw convolution sum is divided by (often the sum of kernel weights) to keep output intensities in range.
Mean (Averaging) filter	A smoothing kernel with equal weights that blurs an image by local averaging.
Laplacian filter	An edge-detection kernel approximating the second derivative of intensity; highlights areas of rapid change.
Sharpening filter	A kernel that boosts the centre pixel relative to its neighbours, accentuating edges and fine detail.
Prewitt (gradient) kernel	A directional kernel that approximates the first derivative of intensity to highlight edges in one direction.
Gaussian filter	A smoothing kernel whose weights follow a Gaussian (bell-curve) distribution, giving a more natural blur than a mean filter.

Step-by-Step Working

1. Import NumPy, OpenCV and Matplotlib.
2. Define a small 5×5 sample image matrix of pixel intensities.
3. Display the sample image using Matplotlib's grayscale colormap.
4. Pad the image with a 1-pixel border (zero padding here; the earlier exploratory step also shows constant-value padding) so every pixel has a full 3×3 neighbourhood.
5. Write a generic apply_filter() function that: pads the image, slides a 3×3 window across every pixel, multiplies the window element-wise with the kernel, sums the result, divides by the given scale factor, and stores it in the output array.
6. Define five 3×3 kernels: Mean, Laplacian, Sharpening, Prewitt (gradient), and Gaussian.
7. Apply apply_filter() with each kernel (scale = 9 for all, matching the sum of the Mean/Prewitt/Laplacian-family kernels used) and display each resulting filtered image.
8. Compare the five outputs to observe smoothing versus edge-highlighting versus sharpening effects.

Python Implementation

1. Sample image and the padding/convolution function:

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

# 5x5 sample "image" of pixel intensities
image = np.array([
    [10, 20, 30, 40, 50],
    [25, 65, 24, 75, 95],
    [45, 65, 75, 95, 100],
    [35, 75, 41, 25, 96],
    [10, 20, 30, 40, 80]
], dtype=float)
```

```
print(image)
plt.imshow(image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Original 5x5 Image")
plt.show()
```

2. Generic 2D convolution with zero-padding:

```
def apply_filter(image, kernel, scale=1):
    rows, cols = image.shape
    padded = np.pad(image, pad_width=1, mode='constant', constant_values=0)
    output = np.zeros((rows, cols))

    for i in range(rows):
        for j in range(cols):
            region = padded[i:i + 3, j:j + 3]
            value = np.sum(region * kernel)
            output[i, j] = value / scale

    return output
```

3. Define the five kernels:

```
mean_kernel  = np.array([[1, 1, 1], [1, 1, 1], [1, 1, 1]], dtype=float)
lap_kernel   = np.array([[0, -1, 0], [-1, 4, -1], [0, -1, 0]], dtype=float)
sharp_kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]], dtype=float)
prewitt_kernel = np.array([[-1, -1, -1], [0, 0, 0], [1, 1, 1]], dtype=float)
gauss_kernel = np.array([[1, 2, 1], [2, 4, 2], [1, 2, 1]], dtype=float)
```

4. Apply each kernel and display the result:

```
mean_out = apply_filter(image, mean_kernel, scale=9)
plt.imshow(mean_out, cmap='gray', vmin=0, vmax=255)
plt.title("Mean (Averaging) Filter")
plt.axis('off'); plt.show()

lap_out = apply_filter(image, lap_kernel, scale=9)
plt.imshow(lap_out, cmap='gray', vmin=0, vmax=255)
plt.title("Laplacian Filter")
plt.axis('off'); plt.show()

sharp_out = apply_filter(image, sharp_kernel, scale=9)
plt.imshow(sharp_out, cmap='gray', vmin=0, vmax=255)
plt.title("Sharpening Filter")
plt.axis('off'); plt.show()

prewitt_out = apply_filter(image, prewitt_kernel, scale=9)
plt.imshow(prewitt_out, cmap='gray', vmin=0, vmax=255)
plt.title("Prewitt (Gradient) Filter")
plt.axis('off'); plt.show()

gauss_out = apply_filter(image, gauss_kernel, scale=9)
plt.imshow(gauss_out, cmap='gray', vmin=0, vmax=255)
plt.title("Gaussian Filter")
plt.axis('off'); plt.show()
```

Explanation of the Code

- The 5×5 NumPy array stands in for a tiny grayscale image so the convolution maths can be followed by hand and verified against the printed output.
- np.pad(..., mode='constant', constant_values=0) surrounds the image with a 1-pixel border of zeros, turning the 5×5 image into 7×7 so every original pixel has a complete 3×3 neighbourhood for the kernel to slide over.
- apply_filter() implements convolution manually: for every output pixel (i, j) it extracts the corresponding 3×3 window from the padded image, multiplies it element-wise with the kernel, sums the result, and divides by scale to keep values in a sensible range.
- The Mean kernel (all ones) averages the neighbourhood, producing a smoothed/blurred output where sharp intensity changes are softened.
- The Laplacian kernel subtracts the four neighbours from four times the centre pixel, producing near-zero output in flat regions and large positive/negative values at edges — this is why the Laplacian output highlights outlines.
- The Sharpening kernel is the Laplacian kernel with 1 added at the centre; it adds the edge-highlighting effect back onto the original pixel value, making edges appear more pronounced while the rest of the image stays recognisable.
- The Prewitt-style kernel (top row -1, bottom row +1) approximates the vertical intensity gradient, so it produces large values where intensity changes from top to bottom — useful for detecting horizontal edges.
- The Gaussian kernel weights the centre pixel most and corners least, giving smoothing that preserves edges better than the uniform Mean kernel.

Expected Output

The 5×5 matrix and its five filtered versions are displayed as grayscale images:

```
[[ 10.  20.  30.  40.  50.]
 [ 25.  65.  24.  75.  95.]
 [ 45.  65.  75.  95. 100.]
 [ 35.  75.  41.  25.  96.]
 [ 10.  20.  30.  40.  80.]]

[Original 5x5 image displayed as a grayscale plot]
[Mean filter output: a visibly smoothed/blurred version of the image]
[Laplacian filter output: near-zero (dark) flat regions with bright/dark
 spikes at intensity transitions, highlighting edge locations]
[Sharpening filter output: original detail preserved with edges emphasised]
[Prewitt filter output: strong response where intensity changes vertically]
[Gaussian filter output: a softer, more natural blur than the Mean filter]
```

Conclusion

The convolution operation was implemented from first principles using zero-padding and a sliding 3×3 window, and five standard kernels were applied to demonstrate the two broad families of spatial filters — smoothing filters (Mean, Gaussian) that reduce noise and detail, and sharpening/edge filters (Laplacian, Sharpening, Prewitt) that emphasise intensity transitions. Understanding this manual implementation clarifies what built-in OpenCV functions such as cv2.filter2D(), cv2.GaussianBlur() and cv2.Laplacian() do internally.